

How Much of a Decision Tree Does Network Traffic Use?

Leaf locality and demand-paged inference in a programmable switch

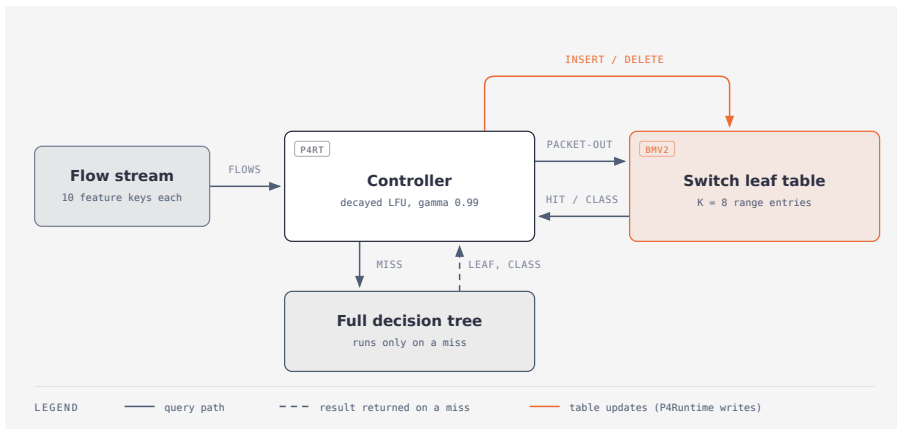
Philip Akekudaga

Machine Learning course project, Fall 2026

The problem

- ▶ A programmable switch can classify flows with a decision tree while forwarding them.
- ▶ Each leaf of a tree is one box of feature space, so one range-match table entry can hold one leaf.
- ▶ Switch tables are small. A tree trained on 22.3M TON_IoT flows has up to about 1,900 leaves.
- ▶ **Question:** does the traffic use only a small, slowly changing set of leaves, so that the switch holds just those and a server handles the rest?

The idea: leaves as pages



- ▶ The switch holds $K = 8$ leaves. A miss goes to the controller, which runs the full tree and rewrites the table.
- ▶ The leaves of one tree never overlap, so the switch answer always equals the full tree's answer.

Data and evaluation

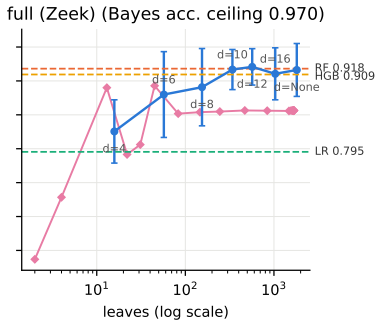
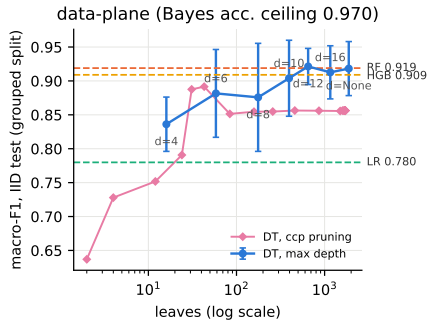
- ▶ TON_IoT processed network flows: 22.3M flows, 96.4% attack, attacks in day-long phases (23 to 30 April 2019).
- ▶ Features a switch can count: bytes, packets, duration, protocol, connection state. No IPs or ports.
- ▶ **The split matters.** Floods repeat identical flows: under a random split, 59% of test flows are copies of training flows.
- ▶ Grouped split: each distinct feature vector lives in one partition. Forward split: train on the first half of each day, replay the second half.

RO1: leakage inflates accuracy

model (data-plane features)	random split	grouped split
decision tree	0.967	0.918
random forest	0.967	0.919
gradient boosting	0.962	0.909
logistic regression	0.866	0.780

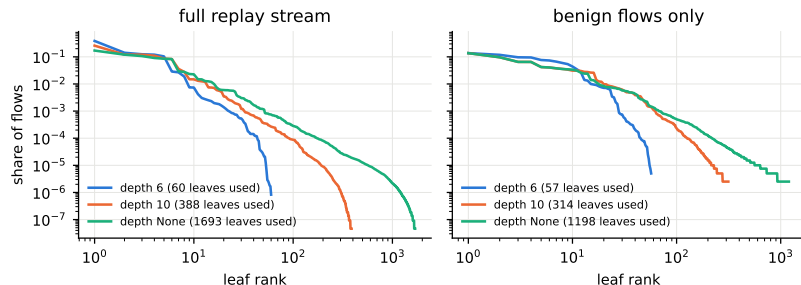
- ▶ Macro-F1, mean over five seeds. Adding IPs and ports gives 0.9999: the tree learns the attacker hosts.
- ▶ On the time-ordered replay (96% attacks), macro-F1 drops to 0.62 (tree) and 0.83 (boosting); always-attack scores 0.49.

RO3: accuracy plateaus, the working set keeps growing



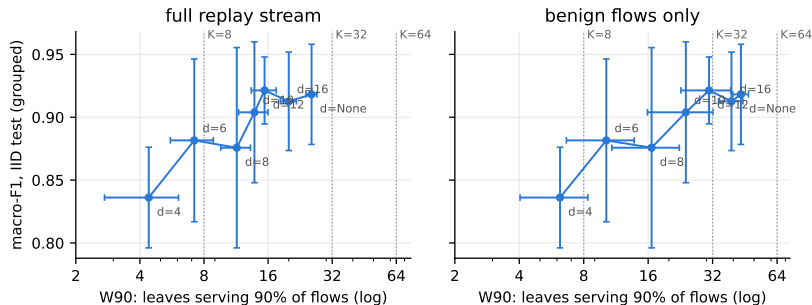
- ▶ Grouped split: macro-F1 0.904 at depth 10, 0.921 at depth 12, 0.918 unpruned (1,880 leaves).
- ▶ Under the random split the curve keeps rising, because deeper trees memorize duplicates.

RO2: which leaves does the traffic use?



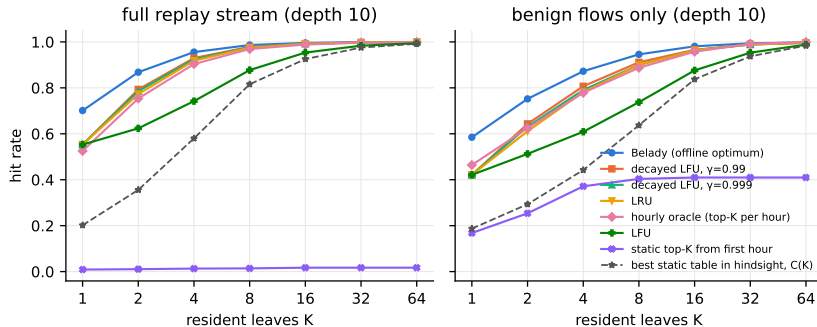
- ▶ Unpruned tree: 26 leaves serve 90% of all flows. Per attack type 2 to 18 leaves; benign traffic needs 43.
- ▶ An 8-entry LRU cache serves 93.6% of flows; the best fixed 8-leaf table in hindsight serves 70.9%.

RO3: size against working set



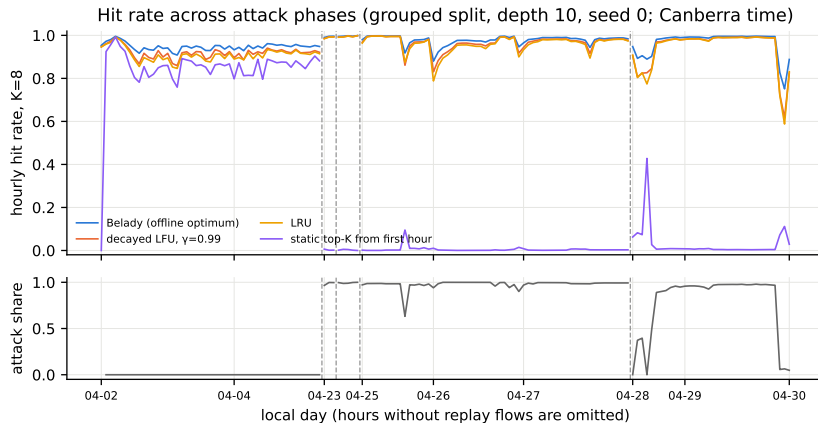
- Depths 4, 6, 10 and 12 are Pareto-optimal. Depth 16 and the unpruned tree cost more entries for no measurable accuracy.

RO4: replacement policies



- ▶ Unpruned tree, 8 entries: decayed LFU 94.7%, Belady (offline optimum) 96.4%, LRU 93.6%, LFU 77.8%.
- ▶ A static table filled from the first hour serves 1.3%: it was chosen weeks before the attacks.

RO4: adapting to attack phases



- After a phase change, decayed LFU needs on average 119 extra misses in the first 10,000 flows; LRU needs 836.

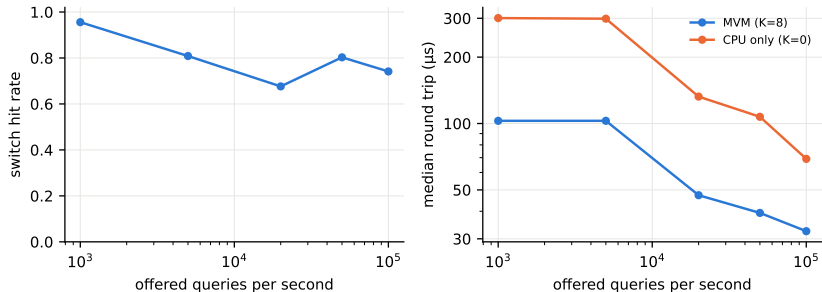
BMv2 prototype

- ▶ Exact encoding: each float32 feature becomes an order-preserving 32-bit key, so range matches reproduce the tree exactly.
- ▶ 60,000 replayed queries: served prediction equals the full tree 100% of the time, and the hit sequence equals the simulator.
- ▶ Hit rate 92.2% (all traffic) and 85.6% (benign). Median hit 0.21 ms, median miss 0.85 ms on BMv2.
- ▶ BMv2 took 3,866 single-entry writes per second, 33,622 per second batched (software switch).

Tofino-1 hardware: MVM with real servers

- ▶ UfiSpace S9180-32X (Tofino-1) with Vision (queries) and Hulk (CPU backend) on 25G links.
- ▶ Tofino range keys are limited to about 20 bits, so each feature is turned into a thermometer code first; the leaf table is ternary over the codes, still one entry per leaf. The depth-10 tree (394 leaves, 361-bit key) is the deepest that fits.
- ▶ 36 runs, 1.08M queries: every query answered, and every answer equals the full tree.
- ▶ Hit rate at 1k queries/s: 95.6% (all traffic) and 84.6% (benign), against 96.3% and 88.5% for the ideal policy; the controller needs 6 ms to install a leaf.

Tofino-1 hardware: switch against CPU



- ▶ Switch pipeline: 362 ns per hit. Round trip at Vision: 102.5 μ s for a switch hit, 300.7 μ s when every query goes to the CPU server.
- ▶ The CPU computes a query in 30 ns; the saving comes from the network hop and host stacks, not from compute.
- ▶ At higher load the hit rate falls to 68 to 81%: control-plane lag spans more queries.

Limitations and next steps

- ▶ Training pools are drawn from the full set with the official class mix; the official train/test file was not available.
- ▶ One testbed, 96% attack traffic: benign locality in a real network may differ.
- ▶ Validation selection of tree depth is unstable across seeds (unpruned twice; depths 10, 12, 16 once each).
- ▶ Control-plane lag costs up to 28 points of hit rate at high load on Tofino-1.
- ▶ Next: official training file, a second dataset (NF-ToN-IoT v3), and replacement decisions closer to the data plane.

Summary

- ▶ Duplicate leakage, not model capacity, made deeper trees look better.
- ▶ Beyond about 650 leaves, accuracy stops improving while the working set keeps growing.
- ▶ Attack traffic is cheap to cache; benign traffic sets the table size.
- ▶ An 8-entry table with decayed LFU serves 94.7% of flows, 1.7 points below the offline optimum.
- ▶ On Tofino-1 the switch answers exactly like the full tree, in 362 ns per hit, at one third of the CPU path's round trip.